The Journal of Open Source Software

pyharm: Analysis and Plotting for GRMHD

Cora Prather [1¶]

1 Black Hole Initiative, Harvard University ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 01 August 2026
Published: unpublished

Summary

pyharm is a scalable python library for plotting and analyzing some general-relativistic magnetohydrodynamic (GRMHD) simulations, especially those simulating matter around black holes. It handles reading a number of different simulation snapshot formats, plotting basic and derived fluid properties in different spacetimes and coordinate systems, and computing arbitrary reductions across large datasets of fluid snapshots.

Statement of need

As large, complex 3D datasets, GRMHD simulation snapshots require sophisticated post-processing and plotting code in order to correctly interpret their data values. Writing new plotting scripts from scratch takes time and effort, not to mention being bug-prone due to the overall complexity. A framework which pre-defines variables and coordinates consistently saves time in understanding these datasets, and improves the quality of results by allowing fixes to be preserved in all future calculations.

State of the field

Many plotting libraries and utilities exist for visualizing and analyzing eulerian hydrodynamics simulations at scale, e.g. (Turk et al., 2010), (Bethel et al., 2012), (Ahrens et al., 2005). However, GRMHD in particular presents two unique challenges that make these tools difficult to use or wholly insufficient:

1. An analytic transformation is often made from a "natural" coordinate system (r, θ, ϕ in the spacetime) into a separate coordinate system where the simulation grid is regular. Imaging simulations requires performing the reverse transformation.
2. Fluid quantities in GRMHD do not generally match the quantities in Newtonian MHD, and generally require the local metric information, and potentially frame transformations, to compute.

Plotting scripts exist which natively support GRMHD grids and quantities, but are generally either private, or specialized to a particular code or style of GRMHD (Bozzola, 2021). pyharm attempts to support as many codes as possible.

Software design

The central class in pyharm is the `FluidState`, representing a snapshot of a GRMHD simulation at some point in time (either generated, memory-backed, or file-backed, generally the latter). Data retrieval and manipulation are both done by means of the `__getitem__` function, allowing users to treat the object as a dictionary containing any desired derived variable. When a key is retrieved, e.g. `dump['beta']` for the plasma β parameter (ratio of gas pressure to

magnetic pressure), it is computed on the fly, returned, and cached in memory for the next use. Quantities can be defined in terms of one another recursively, and thus the code avoids repeating common calculations.

While `pyharm` is designed first as a python library for user scripts, it has gained a number of useful command-line utilities, two of which are easily extensible and broadly useful as a way of accessing `pyharm`'s functionality.

`pyharm movie` handles slice and average plots per-snapshot across a simulation (including optionally encoding the results using `ffmpeg`). Large jobs consisting of many frames can be parallelized across multiple cores or nodes with Python's built-in `multiprocessing` or with `mpi4py.futures` (Rogowski et al., 2023). New movies can be defined solely by adding a function in `phyarm/plots/figures.py` which creates one frame, with the parallelism features shared between all functions.

`pyharm analysis` handles reductions over snapshots from an entire simulation (slices, time-averages, flow rates, etc.). Results are written to a single HDF5 file collecting from all processes/ranks, in a standard layout readable by another command-line script, `pyharm plot-result`.

In addition to the main scripts, `pyharm` provides a number of small utilities for checking simulation logs for progress and errors, checking validity of simulation snapshots, or checking snapshots against one another.

Implementation notes

`pyharm` is written in pure Python using `numpy`, `scipy`, and `matplotlib`. As it is implemented purely with array operations, not loops, the code shares `numpy` and `scipy`'s good efficiency on single CPU cores. Parallelizing operations on a single dump is not a goal of the code – usually, compute-intensive operations are conducted across many dump files in a way which is substantially or wholly embarrassingly parallel. Likewise, GPUs are not a priority as many operations are already significantly bottlenecked by file read speed. Instead, `pyharm` achieves good speed mostly by avoiding unnecessary read and compute operations.

As its most important optimization, `pyharm` supports slicing of HDF5-based dump files without first reading the entire grid into memory. Thus if only, e.g., a slice at $\phi = 0$ is required, `pyharm` supports a slicing operation `sliced_dump = dump[:,:,0]` before any reads are performed, and will read only the given slice from the data file. Most operations are supported on sliced files, including the calculation of derived variables, reductions, and plotting.

In addition, the (sliced) arrays corresponding to each key are natively cached by the pyharm data objects: file readers can cache the variables retrieved by a `FluidState` object, and `FluidState` objects themselves can cache the results of computations to avoid repeating them. Caching can be memory-intensive, but when combined with slicing it can lead to dramatic improvements in runtime of complex analyses and reductions which share basic components.

With these optimizations, a basic variable from a low-resolution (192x96x96 zones) run of 2000 snapshot files can be plotted in as little as a minute on a 12-core desktop computer, and a single node of Harvard Cannon (56 cores) images a more substantial simulation (192x96x96 zones, ~3000 snapshots) in about a minute as well – in each case, more than enough to generate a 30fps movie as it plays. A much more complex analysis (about 70 reductions over derived variables) completes in 20 minutes on the desktop and 31 minutes on the cluster.

Tests

As primarily a plotting framework, most bugs or undesired behaviors in pyharm cannot be easily tested in an automated way. However, calculations of physical quantities are tested

against expectations, such as an example calculation of 4-vector quantities with known results. "Golden file" regression tests of the calculation of basic reductions are also available, over an array of different file formats.

Research impact statement

pyharm has been used in obtaining the results of many published papers, especially from the GRMHD code KHARMA (Prather, 2025). This includes collaboration papers from the EHT such as (Event Horizon Telescope Collaboration et al., 2019) and (Event Horizon Telescope Collaboration et al., 2022), and independent papers using KHARMA, e.g. most recently (Thomas et al., 2026), (Bukowiecka et al., 2026) and (Stanway et al., 2026).

Acknowledgements

Work supported by the Black Hole Initiative at Harvard University, which is funded in part by the Gordon and Betty Moore Foundation (grant #13526). It was also made possible through the support of a grant from the John Templeton Foundation (grant #63445). The opinions expressed in this publication are those of the author(s) and do not necessarily reflect the views of these Foundations.

An award of computer time was provided by the U.S. Department of Energy's (DOE) Innovative and Novel Computational Impact on Theory and Experiment (INCITE) Program. This research used supporting resources at the Argonne and the Oak Ridge Leadership Computing Facilities. The Argonne Leadership Computing Facility at Argonne National Laboratory is supported by the Office of Science of the U.S. DOE under Contract No. DE-AC02-06CH11357. The Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory is supported by the Office of Science of the U.S. DOE under Contract No. DE-AC05-00OR22725.

This work used Delta CPU at the National Center for Supercomputing Applications through allocation PHY250339 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

AI usage disclosure

No AI tools were used in developing pyharm or this paper.

References

Ahrens, J., Geveci, B., & Law, C. (2005). ParaView: An End-User Tool for Large-Data Visualization. In *Visualization Handbook* (pp. 717–731). Elsevier. https://doi.org/10.1016/B978-012387582-2/50038-1

Bethel, E. W., Childs, H., & Hansen, C. (Eds.). (2012). *High Performance Visualization: Enabling Extreme-Scale Scientific Insight* (0th ed.). Chapman and Hall/CRC. https://doi.org/10.1201/b12985

Bozzola, G. (2021). Kuibit: Analyzing Einstein Toolkit simulations with Python. *Journal of Open Source Software*, *6*(60), 3099. https://doi.org/10.21105/joss.03099

Bukowiecka, N., Ricarte, A., Kocherlakota, P., & Prather, C. (2026). Observational distinguishability of the Kerr and Kerr-Hayward metrics to EHT. *Physical Review D*, *114*(2), 023008. https://doi.org/10.1103/rfrx-tfz8

Event Horizon Telescope Collaboration, Akiyama, K., Alberdi, A., Alef, W., Algaba, J. C., Anantua, R., Asada, K., Azulay, R., Bach, U., Baczko, A.-K., Ball, D., Baloković, M., Barrett, J., Bauböck, M., Benson, B. A., Bintley, D., Blackburn, L., Blundell, R., Bouman, K. L., … White, C. (2022). First Sagittarius A* Event Horizon Telescope Results. V. Testing Astrophysical Models of the Galactic Center Black Hole. *The Astrophysical Journal Letters*, *930*(2), L16. https://doi.org/10.3847/2041-8213/ac6672

Event Horizon Telescope Collaboration, Akiyama, K., Alberdi, A., Alef, W., Asada, K., Azulay, R., Baczko, A.-K., Ball, D., Baloković, M., Barrett, J., Bintley, D., Blackburn, L., Boland, W., Bouman, K. L., Bower, G. C., Bremer, M., Brinkerink, C. D., Brissenden, R., Britzen, S., … Zhang, S. (2019). First M87 Event Horizon Telescope Results. V. Physical Origin of the Asymmetric Ring. *The Astrophysical Journal*, *875*, L5. https://doi.org/10.3847/2041-8213/ab0f43

Prather, C. (2025). KHARMA: Flexible, Portable Performance for GRMHD. In C. Bambi, Y. Mizuno, S. Shashank, & F. Yuan (Eds.), *New Frontiers in GRMHD Simulations* (pp. 167–201). Springer Nature Singapore. https://doi.org/10.1007/978-981-97-8522-3_5

Rogowski, M., Aseeri, S., Keyes, D., & Dalcin, L. (2023). Mpi4py.futures: MPI-Based Asynchronous Task Execution for Python. *IEEE Transactions on Parallel and Distributed Systems*, *34*(2), 611–622. https://doi.org/10.1109/TPDS.2022.3225481

Stanway, J. S., Prather, C., Ward-Thompson, D., Walton, T. J., Patterson, B., & Cho, H. (2026). *Supermassive Black Holes: Modelling strongly and weakly magnetised misaligned accretion disks* (No. arXiv:2605.27334). arXiv. https://doi.org/10.48550/arXiv.2605.27334

Thomas, T. A., Ricarte, A., Prather, C., & Cho, H. (2026). Observational Properties of Near-maximally Spinning Supermassive Black Holes. *Publications of the Astronomical Society of the Pacific*, *138*(6), 064101. https://doi.org/10.1088/1538-3873/ae7902

Turk, M. J., Smith, B. D., Oishi, J. S., Skory, S., Skillman, S. W., Abel, T., & Norman, M. L. (2010). Yt: A MULTI-CODE ANALYSIS TOOLKIT FOR ASTROPHYSICAL SIMULATION DATA. *The Astrophysical Journal Supplement Series*, *192*(1), 9. https://doi.org/10.1088/0067-0049/192/1/9